

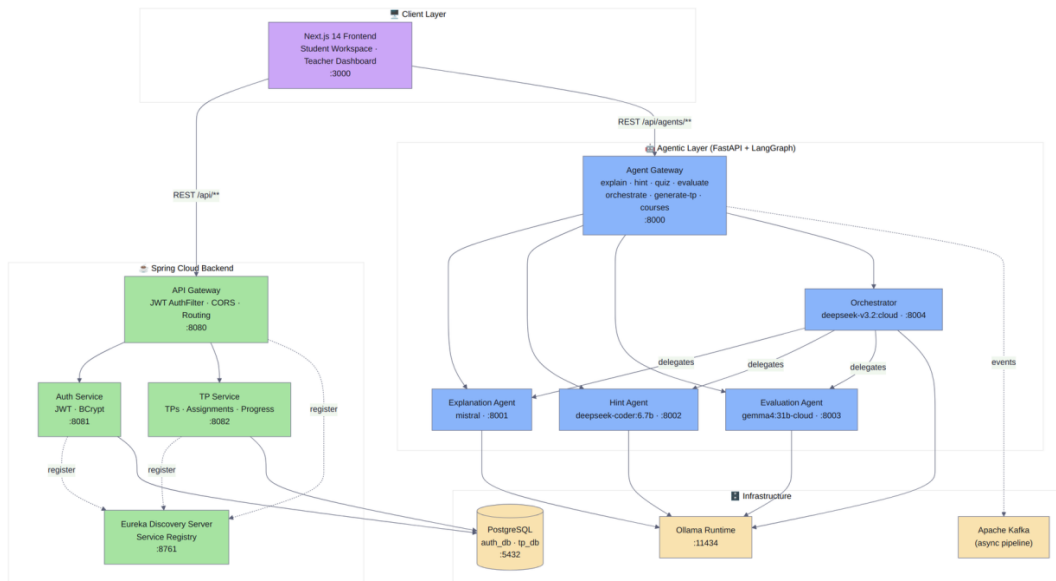


ECOLE NORMALE SUPÉRIEURE DE L'ENSEIGNEMENT
TECHNIQUE DE MOHAMMEDIA
UNIVERSITÉ HASSAN II DE CASABLANCA

RAPPORT DE PROJET

Agentic TP Platform

Plateforme éducative pilotée par des agents d'intelligence artificielle
pour l'encadrement des Travaux Pratiques de programmation



Architecture générale de la plateforme Agentic TP

Équipe	Anass Makhlouk — Houdaifa Bahou — Badr Eddine Toubani
Contexte	Projet de fin de module — Web et Web Sémantique
Démo en ligne	tp-front-seven.vercel.app
Dépôt de code	github.com/embedding3x/enset-challenge-submission-embedding3x

Sommaire

Sommaire.....	2
1. Introduction.....	4
1.1. Contexte.....	4
1.2. Objectifs du projet.....	4
1.3. Points forts de la solution	5
2. Cahier des charges.....	6
2.1. Problématique.....	6
2.2. Besoins identifiés.....	6
2.2.1. Besoins fonctionnels	6
2.2.2. Besoins non fonctionnels	7
2.3. Solutions proposées.....	7
3. Conception fonctionnelle	7
3.1. Identification des acteurs.....	8
3.2. Diagramme de cas d'utilisation	8
3.3. Diagramme de classes des agents.....	9
3.4. Modèle de données	9
3.5. Boucle ReAct de raisonnement (LangGraph)	10
3.6. Graphe de routage de l'orchestrateur.....	11
3.7. Diagramme de séquence — session étudiant.....	11
3.8. Patrons de conception mobilisés	12
3.8.1. Patron Interprète (Interpreter).....	12
3.8.2. Patron Stratégie et repli gracieux	12
4. Conception technique	13
4.1. Architecture générale en couches.....	13
4.2. Pile technologique	14
4.3. La couche agentic : quatre agents spécialisés.....	14
4.3.1. Agent Explication — phase d'accueil	15
4.3.2. Agent Indices — phase de codage	15
4.3.3. Agent Évaluation — évaluation post-session.....	15
4.3.4. Orchestrateur — routage multi-étapes	15
4.4. Backend microservices (Spring Cloud)	16

4.5. Agent Gateway (FastAPI)	16
4.6. Frontend Next.js	16
4.7. Mécanismes anti-triche	17
4.8. Infrastructure et déploiement	17
5. Réalisation	18
5.1. Page d'accueil et authentification	18
5.2. Espace enseignant	19
5.3. Espace étudiant	22
5.4. Scénarios d'utilisation	23
5.4.1. Parcours étudiant en trois phases	23
5.4.2. Parcours enseignant.....	24
5.5. État d'avancement.....	24
6. Conclusion	25
6.1. Résumé du projet	25
6.2. Défis rencontrés et limites actuelles	25
6.3. Recommandations et perspectives	25
6.3.1. Court terme.....	25
6.3.2. Moyen terme	26
6.4. Innovations clés.....	26

1. Introduction

Ce rapport présente la conception et la réalisation d'**Agentic TP Platform**, une plateforme éducative web dont l'objectif est d'accompagner les étudiants tout au long de leurs Travaux Pratiques (TP) de programmation grâce à une couche d'agents d'intelligence artificielle spécialisés. Il décrit le contexte qui a motivé le projet, ses objectifs, le cahier des charges retenu, la conception fonctionnelle et technique de la solution, puis les fonctionnalités effectivement développées, avant de conclure sur les défis rencontrés et les pistes d'amélioration.

1.1. Contexte

Les Travaux Pratiques de programmation occupent une place centrale dans la formation en informatique : c'est en écrivant du code, en se trompant et en corrigeant ses erreurs que l'étudiant assimile réellement les concepts enseignés en cours. Pourtant, l'encadrement de ces séances reste difficile à mettre en œuvre à grande échelle. Un enseignant seul ne peut pas accompagner individuellement chaque étudiant au moment précis où celui-ci bloque, et la correction manuelle des rendus, étape par étape, est extrêmement chronophage. Les étudiants les plus en difficulté, qui auraient le plus besoin d'une aide ciblée, sont souvent ceux qui osent le moins solliciter l'enseignant.

À cette difficulté structurelle s'ajoute un phénomène récent : la généralisation des assistants de codage généralistes a rendu le copier-coller de solutions complètes à la fois trivial et indétectable. Un étudiant peut désormais obtenir en quelques secondes un code fonctionnel sans avoir compris la démarche qui y conduit, ce qui vide l'exercice de sa valeur pédagogique et fausse l'évaluation. Le défi n'est donc plus seulement d'aider l'étudiant, mais de l'aider sans faire le travail à sa place.

La plateforme **Agentic TP** a été conçue pour répondre simultanément à ces deux enjeux. Elle combine un espace de travail web moderne, une architecture en microservices Spring Cloud et une couche d'agents d'IA spécialisés — chacun animé par un modèle de langage local distinct servi via Ollama — afin d'offrir un environnement de codage à la fois guidé, intelligent et académiquement honnête. L'assistance y est pensée comme un accompagnement socratique : l'IA oriente, questionne et débloque, mais ne livre jamais la solution toute faite.

1.2. Objectifs du projet

Le projet poursuit cinq objectifs principaux, qui structurent l'ensemble des choix de conception présentés dans ce rapport :

- **Concevoir une couche agentique modulaire** où chaque agent est spécialisé par tâche et animé par le modèle de langage le mieux adapté à cette tâche (un modèle de raisonnement pour le routage, un modèle de code pour les indices, etc.).
- **Mettre en place une architecture microservices robuste** couvrant l'authentification par jetons JWT, la gestion des TP et de la progression, une passerelle d'API et un registre de services pour la découverte dynamique.

- **Développer un espace de travail étudiant complet** intégrant un éditeur de code, un aperçu HTML en direct et des mécanismes anti-triche intégrés à l'éditeur lui-même.
- **Offrir à l'enseignant un tableau de bord de pilotage** permettant de créer, d'assigner et de suivre les TP, ainsi qu'un générateur de TP assisté par IA pour réduire la charge de préparation.
- **Conteneuriser l'ensemble de la solution** afin de garantir un déploiement reproductible, lançable en une seule commande, aussi bien en développement qu'en production.

1.3. Points forts de la solution

Une démonstration en ligne de la plateforme est accessible à l'adresse tp-front-seven.vercel.app. Les éléments qui distinguent la solution des outils d'assistance classiques sont résumés dans le tableau ci-dessous.

Atout	Description
Agents IA spécialisés	Trois agents dédiés — Explication, Indices et Évaluation — chacun doté de son propre modèle, de son prompt système et de son jeu d'outils, coordonnés par un orchestrateur.
Modèles 100 % locaux	Les LLM (mistral, deepseek-coder, gemma4, deepseek-v3.2) sont servis localement via Ollama, garantissant la confidentialité des données ; un relais cloud reste optionnel.
Intégrité académique	Copier-coller désactivé, minuteur de session, et validation automatique des balises HTML réellement écrites par l'étudiant.
Pilotage enseignant	Suivi de la progression par étudiant, usage des indices, et score de compréhension issu d'un quiz généré à partir du code rendu.

2. Cahier des charges

Cette section formalise la problématique à laquelle la plateforme répond, en déduit les besoins fonctionnels et non fonctionnels, puis expose les solutions retenues pour y répondre.

2.1. Problématique

La problématique centrale du projet peut s'énoncer ainsi : *comment fournir à chaque étudiant un accompagnement personnalisé et en temps réel pendant ses TP, tout en garantissant que c'est bien lui qui apprend et produit le code, et en donnant à l'enseignant une vision fine et objective de la progression de toute sa classe ?*

Cette question, en apparence simple, met en tension trois exigences qui se contredisent dans les outils existants : aider efficacement tend à donner la réponse (au détriment de l'apprentissage) ; empêcher la triche tend à brider l'assistance ; et mesurer la compréhension réelle suppose de distinguer le code compris du code copié. La plateforme doit concilier ces trois exigences plutôt que d'en sacrifier une.

La problématique se décline ainsi en trois axes complémentaires :

- **Pédagogie active** — guider l'étudiant par la méthode socratique (questions, indices progressifs, reformulation) plutôt que lui fournir directement la solution, afin qu'il construise lui-même son raisonnement.
- **Intégrité académique** — empêcher techniquement le copier-coller de code externe et valider que le code requis a réellement été saisi par l'étudiant, étape par étape.
- **Pilotage enseignant** — mesurer objectivement la compréhension de chaque étudiant au moyen d'un quiz généré à partir de son propre code, et restituer ces données dans un tableau de bord exploitable.

2.2. Besoins identifiés

À partir de la problématique, les besoins ont été répartis entre besoins fonctionnels (ce que le système doit faire) et besoins non fonctionnels (les qualités attendues du système).

2.2.1. Besoins fonctionnels

Acteur	Besoin fonctionnel
Étudiant	S'authentifier, consulter ses TP assignés, recevoir l'explication d'une étape, écrire du code dans un éditeur avec aperçu en direct, demander des indices progressifs, et passer un quiz d'évaluation en fin de session.
Enseignant	S'authentifier, créer un TP (manuellement ou via l'IA), l'assigner à des étudiants avec une échéance, suivre la progression de la classe, indexer des documents de cours (RAG) et consulter les scores et retours générés.
Agents IA	Expliquer une étape sans donner de code, produire des indices à difficulté croissante, générer un quiz ancré dans le code rendu, le corriger, et router les requêtes complexes vers le bon agent.

2.2.2. Besoins non fonctionnels

- **Confidentialité** — le code et les données des étudiants ne doivent pas quitter l’infrastructure ; d’où le choix de modèles de langage exécutés localement.
- **Modularité et évolutivité** — chaque agent et chaque service doit pouvoir évoluer ou être remplacé indépendamment, sans recompiler l’ensemble.
- **Robustesse** — l’expérience ne doit pas s’interrompre si un agent IA est momentanément indisponible (repli gracieux sur une logique déterministe locale).
- **Reproductibilité du déploiement** — l’ensemble doit être conteneurisé et démarrable d’une seule commande, sur n’importe quelle machine.
- **Sécurité** — authentification par jetons signés, mots de passe hachés, et contrôle d’accès par rôle (enseignant / étudiant).

2.3. Solutions proposées

Pour satisfaire ces besoins, la solution articule quatre réponses techniques structurantes, détaillées dans les sections de conception qui suivent :

Besoin	Solution retenue
Accompagnement personnalisé	Une couche de quatre agents IA spécialisés (Explication, Indices, Évaluation, Orchestrateur), bâtis sur le patron ReAct de LangGraph, chacun servi par un modèle adapté à sa tâche.
Intégrité académique	Désactivation du collage, du glisser-déposer et du clic droit dans l’éditeur, minuteur de session, et validation formelle des balises HTML via un patron Interprète.
Pilotage et évaluation	Quiz généré à partir du code réellement soumis, score calculé de façon déterministe, et tableau de bord enseignant restituant progression, temps et indices utilisés.
Robustesse et déploiement	Architecture microservices Spring Cloud avec registre Eureka et passerelle, modèles locaux via Ollama, et conteneurisation complète avec Docker Compose.

3. Conception fonctionnelle

Cette section décrit le système du point de vue de ses usages : qui interagit avec la plateforme, quelles fonctions ils mobilisent, comment les données sont structurées et comment les principaux scénarios se déroulent. Les diagrammes UML retenus (cas d’utilisation, classes, séquence) sont complétés par deux schémas de comportement propres à la couche agentique (boucle ReAct et routage), jugés pertinents pour comprendre le fonctionnement réel du système.

3.1. Identification des acteurs

Trois acteurs interagissent avec la plateforme : deux acteurs humains, qui en sont les utilisateurs directs, et un acteur système constitué par la couche d'agents IA, sollicité par de nombreux cas d'utilisation.

Acteur	Rôle dans le système
Enseignant	Conçoit les TP (manuellement ou avec l'aide de l'IA), les assigne aux étudiants, indexe des supports de cours et suit la progression et les résultats de sa classe. Il pilote la dimension pédagogique de la plateforme.
Étudiant	Consulte ses TP assignés, suit les explications, écrit son code dans l'éditeur intégré, demande des indices et passe le quiz d'évaluation. Il est l'utilisateur final de l'accompagnement guidé.
Agents IA (Ollama)	Acteur système secondaire. Sollicité par les cas d'utilisation d'explication, d'indice, de génération et de correction de quiz, ainsi que de génération de TP. Il n'initie jamais d'action de lui-même.

3.2. Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation ci-dessous synthétise les interactions entre les acteurs et le système. Les cas propres à l'enseignant (créer, assigner, indexer, suivre) et ceux propres à l'étudiant (recevoir une explication, écrire du code, demander un indice, passer le quiz) sont reliés à l'acteur système « Agents IA » à chaque fois qu'une fonction mobilise un modèle de langage.

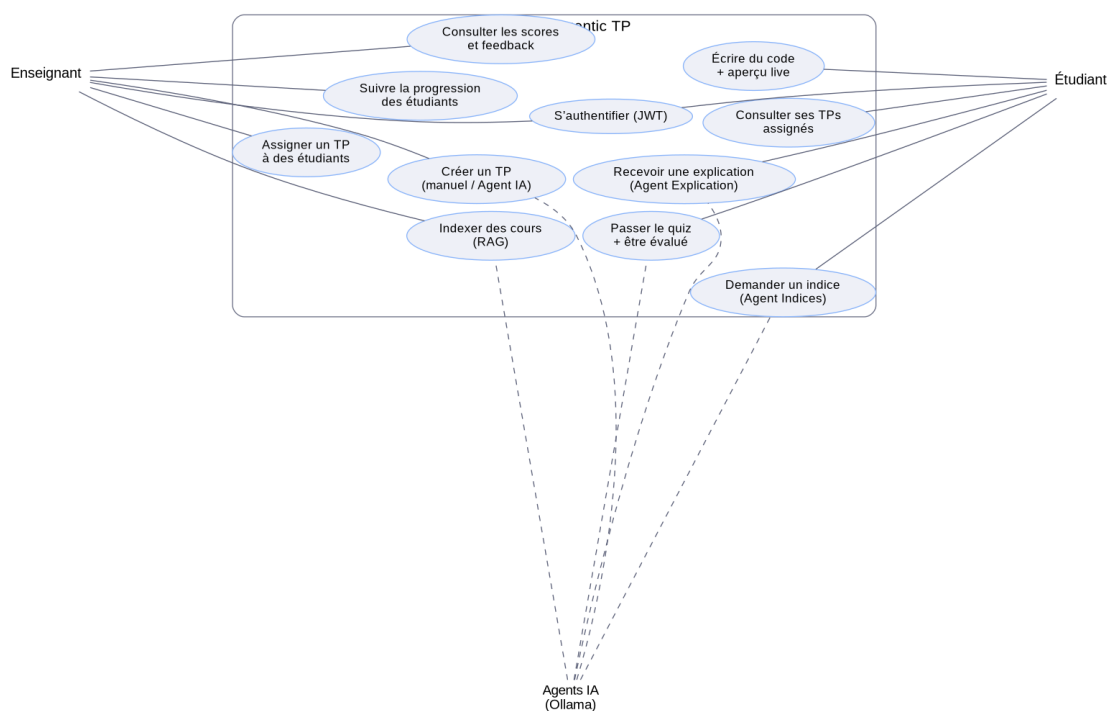


Figure 1 — Diagramme de cas d'utilisation de la plateforme

On note que l'authentification est partagée par les deux acteurs humains, tandis que les cas d'utilisation à forte valeur pédagogique — explication, indice, évaluation — dépendent systématiquement de l'acteur Agents IA, ce qui illustre la place centrale de la couche agentic dans le fonctionnement de la plateforme.

3.3. Diagramme de classes des agents

Tous les agents de la plateforme partagent le même patron structurel, ce qui constitue un choix de conception déterminant : une application FastAPI, un modèle de langage ChatOllama, un agent LangGraph créé via la fonction `create_react_agent`, un prompt système et un jeu d'outils typés. Les agents ne diffèrent que par quatre paramètres : le modèle de langage utilisé, sa température, l'ensemble de ses outils et ses points d'entrée HTTP.

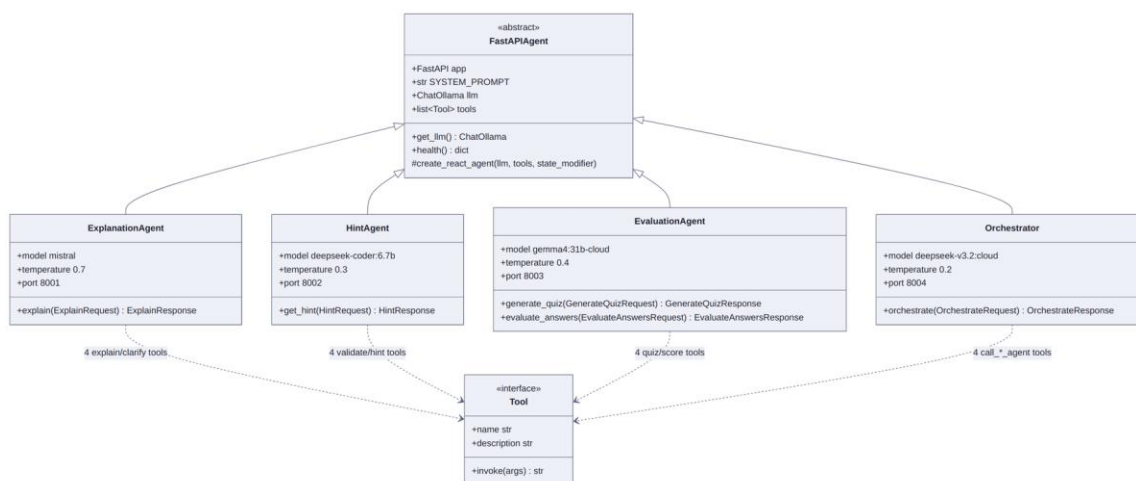


Figure 2 — Diagramme de classes : spécialisation d'un agent FastAPI abstrait

Cette structure commune — un « agent FastAPI abstrait » dont chaque agent concret est une spécialisation — simplifie considérablement la maintenance : ajouter un nouvel agent revient à instancier le même patron avec un modèle et des outils différents, sans réécrire la logique d'orchestration. C'est une application directe du principe de réutilisation par composition.

3.4. Modèle de données

Le modèle métier repose sur quatre entités persistées dans PostgreSQL. Les structures imbriquées — étapes d'un TP, étapes complétées, code soumis — sont stockées dans des colonnes JSON plutôt que dans des tables de jointure dédiées. Ce choix offre la souplesse nécessaire à un schéma de TP variable (un TP peut comporter un nombre quelconque d'étapes, chacune avec ses propres balises requises) sans multiplier les tables ni les migrations.

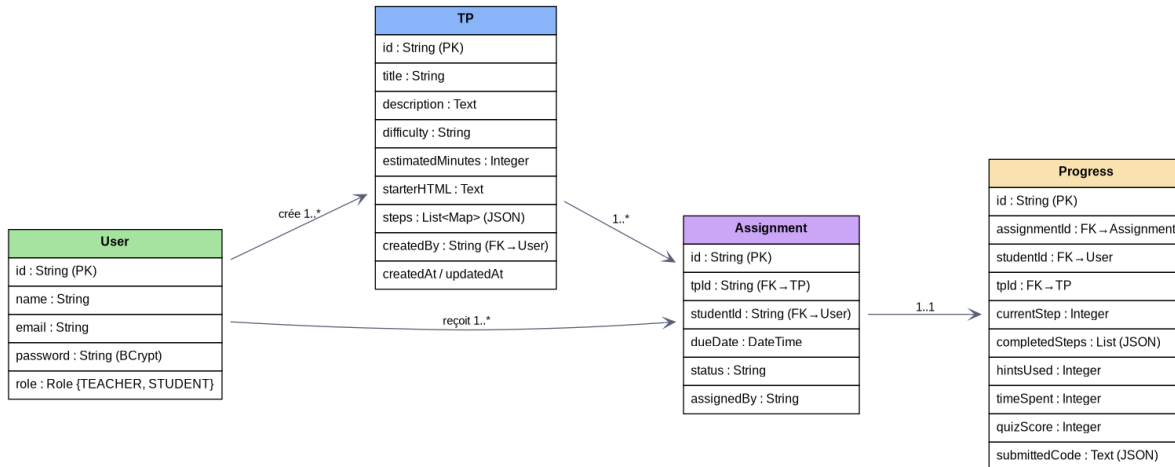


Figure 3 — Modèle de données : entités User, TP, Assignment et Progress

Les entités User et TP sont gérées respectivement par le auth-service et le tp-service, conformément à la séparation des responsabilités entre microservices. Une entité Assignment lie un TP à un étudiant en fixant une échéance, tandis que l'entité Progress suit l'avancement étape par étape, le nombre d'indices consommés, le temps passé et le score final obtenu au quiz. Cette dernière entité est le point de convergence des données exploitées par le tableau de bord enseignant.

3.5. Boucle ReAct de raisonnement (LangGraph)

Au-delà des diagrammes UML statiques, il est utile de représenter le comportement dynamique d'un agent. Chaque agent spécialisé exécute la boucle standard ReAct : raisonner, agir, puis observer. À chaque tour, le modèle de langage choisit soit d'appeler un outil (par exemple valider des balises HTML ou générer un indice), soit d'émettre sa réponse finale ; la boucle se poursuit tant que des appels d'outils sont demandés, puis se termine.

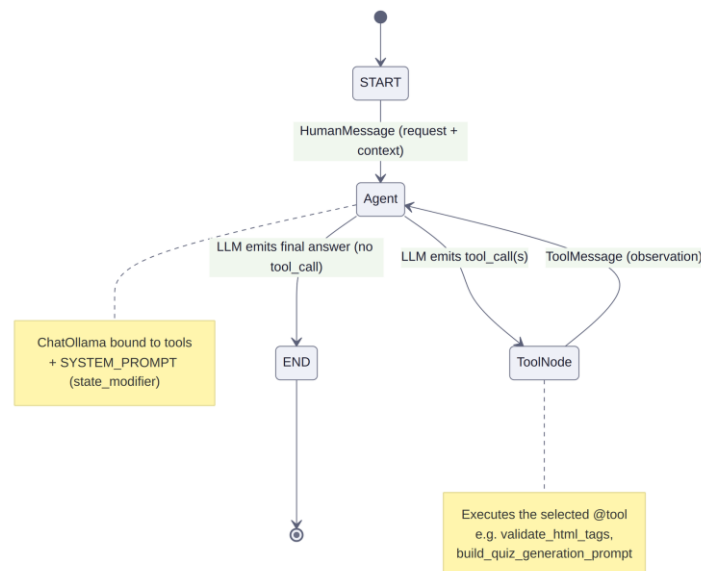


Figure 4 — Boucle ReAct exécutée par chaque agent

Ce mécanisme est ce qui distingue un agent d'un simple appel de modèle : plutôt que de produire un texte en un seul passage, l'agent enchaîne des étapes de raisonnement entrecoupées d'actions concrètes (les outils), ce qui lui permet de fonder ses réponses sur des données vérifiées plutôt que sur sa seule génération.

3.6. Graphe de routage de l'orchestrateur

L'orchestrateur est lui-même un agent ReAct, mais ses outils ne sont pas des fonctions locales : ce sont des appels HTTP vers les autres agents. Le modèle deepseek-v3.2, choisi pour ses capacités de raisonnement, interprète l'intention de la requête entrante, délègue le traitement à l'agent approprié (Explication, Indices ou Évaluation), puis agrège la réponse avant de la renvoyer au client.

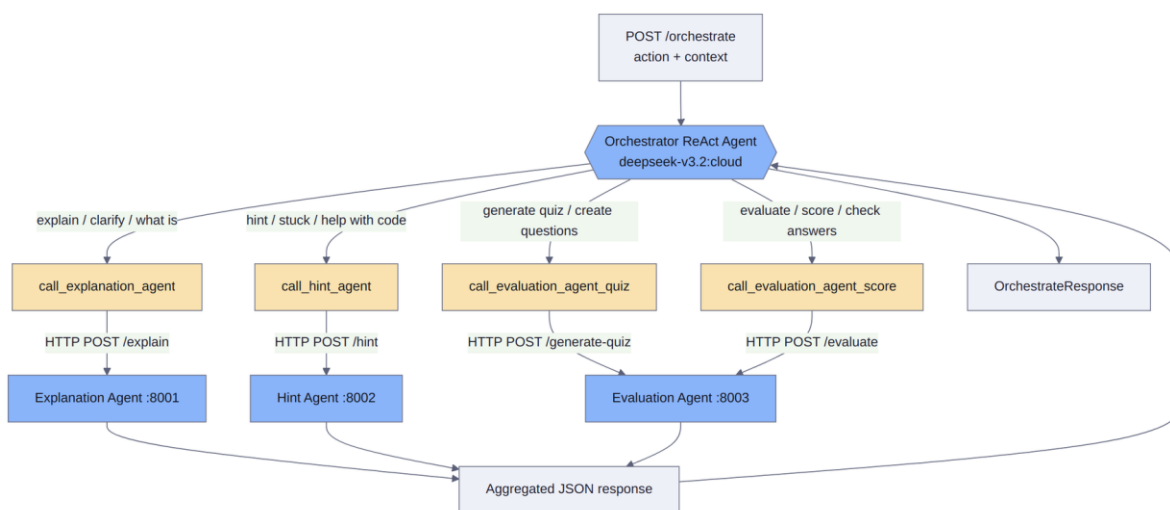


Figure 5 — Routage des requêtes par l'orchestrateur

Ce schéma permet de traiter des requêtes composites qui nécessiteraient autrement plusieurs allers-retours côté client, et centralise la logique de coordination en un seul point clairement identifié.

3.7. Diagramme de séquence — session étudiant

Le scénario nominal d'une session étudiant se déroule en trois phases successives — explication, codage puis évaluation — chacune mobilisant un agent distinct. Toutes les interactions transitent par l'Agent Gateway, qui constitue le point d'entrée unique de la couche IA et masque au client la complexité du routage interne.

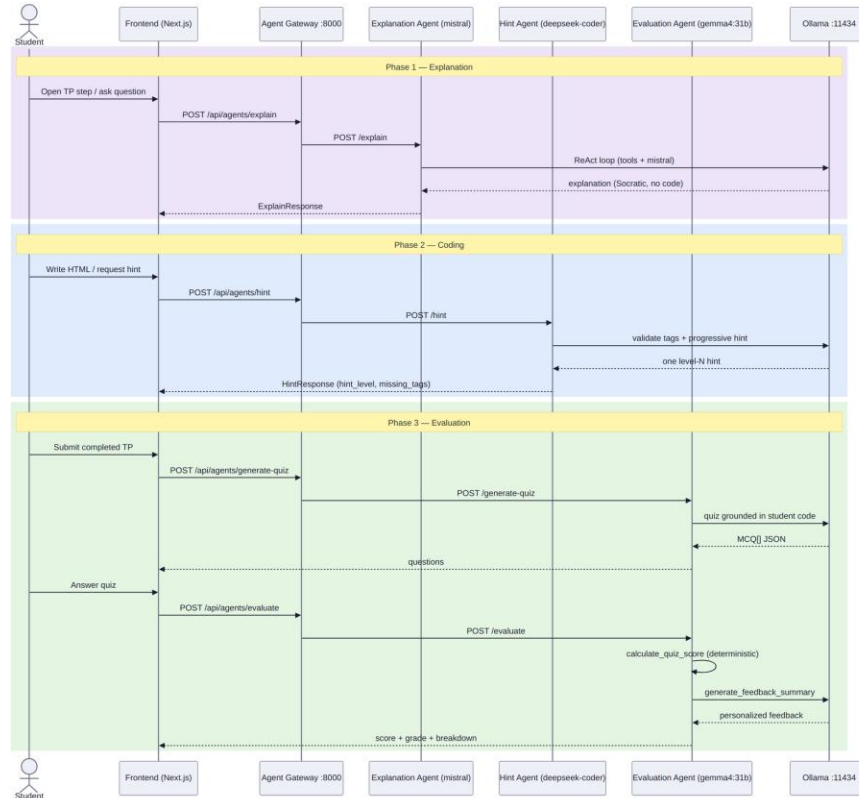


Figure 6 — Diagramme de séquence d'une session étudiant en trois phases

Ce découpage en trois phases reflète directement la pédagogie visée : on explique avant de coder, on aide pendant le codage, et on évalue la compréhension une fois le code produit — jamais l'inverse.

3.8. Patrons de conception mobilisés

3.8.1. Patron Interprète (Interpreter)

La validation des balises HTML requises pour chaque étape est modélisée par un arbre d'expressions composable. Une `TagExpression` vérifie la présence d'une balise donnée dans le code de l'étudiant, et une `AndExpression` compose plusieurs expressions ; la validation d'une étape réussit si, et seulement si, toutes ses sous-expressions sont satisfaites. Ce patron rend la règle de validation déclarative et extensible : ajouter une nouvelle contrainte revient à ajouter une expression à l'arbre.

```

interface HTMLExpression { interpret(html: string): boolean; }

class TagExpression implements HTMLExpression {
  interpret(html) { /* la balise est-elle présente ? */ }
}

class AndExpression implements HTMLExpression {
  interpret(html) {
    return this.expressions.every(e => e.interpret(html));
  }
}
  
```

3.8.2. Patron Stratégie et repli gracieux

Chaque composant front-end faisant appel à l'IA (ExplanationChat, HintBox, QuizComponent) détecte la disponibilité de l'agent correspondant et bascule, en cas d'indisponibilité, sur une logique déterministe locale. Cette stratégie de repli garantit la continuité de l'expérience même lorsqu'un modèle n'est pas joignable : l'étudiant reçoit alors une aide simplifiée mais fonctionnelle, plutôt qu'un message d'erreur.

4. Conception technique

Cette section décrit l'architecture technique de la plateforme et les technologies retenues pour chacune de ses couches, en justifiant les principaux choix d'ingénierie.

4.1. Architecture générale en couches

Le système s'organise en quatre couches faiblement couplées. La couche cliente, développée en Next.js, dialogue d'une part avec la passerelle Spring Cloud pour toutes les données métier (authentification, TP, progression), et d'autre part avec l'Agent Gateway pour l'ensemble des interactions IA. La couche agentique expose chaque agent comme un service FastAPI indépendant, et la couche infrastructure regroupe les briques techniques transverses : Ollama pour les modèles, PostgreSQL pour la persistance et Kafka pour la messagerie asynchrone.

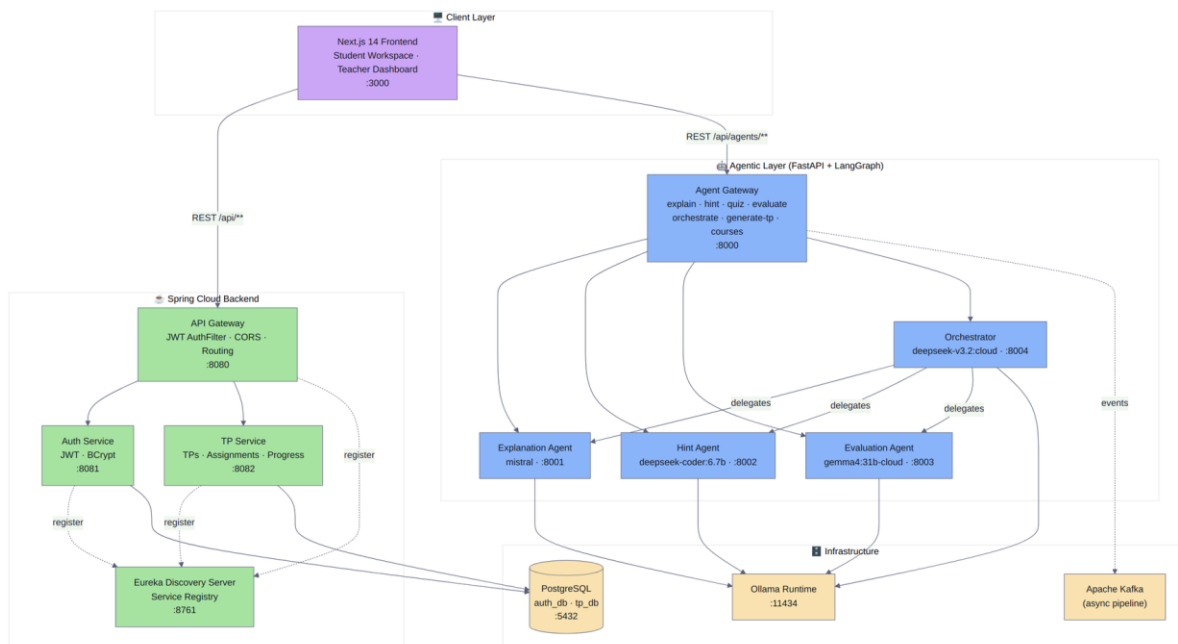


Figure 7 — Architecture en couches : client, backend Spring Cloud, couche agentique et infrastructure

Principe directeur : la séparation stricte entre la passerelle métier (port 8080) et la passerelle IA (port 8000) permet de faire évoluer la couche agentique sans toucher au cœur transactionnel, et réciproquement. Les deux mondes — le transactionnel Java et l'agentique Python — communiquent uniquement par contrats HTTP clairement définis, ce qui autorise des cycles de développement et de déploiement indépendants.

4.2. Pile technologique

Le tableau suivant récapitule les technologies retenues couche par couche, avec les détails d'implémentation les plus significatifs.

Couche	Technologie	Détails
Frontend	Next.js 14 + TypeScript + TailwindCSS	Thème Catppuccin Mocha, éditeur anti-collage, aperçu HTML en direct.
Auth Service	Spring Boot 3.3.4 + Spring Security	JWT (jjwt 0.12), hachage BCrypt, PostgreSQL, client Eureka.
TP Service	Spring Boot 3.3.4 + JPA	TP, assignations, progression ; stockage en colonnes JSON.
API Gateway	Spring Cloud Gateway	Filtre JWT, gestion CORS, routage vers les microservices.
Discovery	Netflix Eureka Server	Registre central des services Spring Cloud.
Framework agents	LangGraph + LangChain Community	Patron create_react_agent avec modèles Ollama.
Serveurs agents	FastAPI + uvicorn	Un service par agent, isolation par environnement virtuel Python.
Fournisseur LLM	Ollama (local + relais cloud)	mistral, deepseek-coder:6.7b, gemma4:31b, deepseek-v3.2.
Base de données	PostgreSQL 16	auth_db et tp_db, colonnes JSON pour les données imbriquées.
Bus de messages	Apache Kafka	Topics définis pour un pipeline asynchrone (optionnel).
Conteneurisation	Docker Compose v2	12 services, réseau partagé, contrôles de santé.

4.3. La couche agentique : quatre agents spécialisés

Tous les agents sont construits avec l'appel `create_react_agent(llm, tools, state_modifier=SYSTEM_PROMPT)`. Chacun se connecte à un modèle local différent via Ollama et expose ses capacités sous forme de fonctions typées (les outils). Le choix d'un modèle distinct par agent est délibéré : il permet d'affecter à chaque tâche le modèle qui la sert le mieux, plutôt que de recourir à un modèle unique généraliste.

Agent	Modèle	Temp.	Port	Rôle
Explication	mistral	0.7	8001	Explique les étapes du TP par la méthode socratique.
Indices	deepseek-coder:6.7b	0.3	8002	Indices progressifs sur 4 niveaux de précision.
Évaluation	gemma4:31b	0.4	8003	Génère et corrige le quiz de compréhension.

Agent	Modèle	Temp.	Port	Rôle
Orchestrateur	deepseek-v3.2	0.2	8004	Routage multi-agents des requêtes composites.

La température décroît à mesure que la tâche exige de la précision : élevée (0,7) pour l'Explication, qui doit varier ses formulations ; basse (0,2–0,4) pour l'Évaluation et l'Orchestrateur, où le déterminisme prime.

4.3.1. Agent Explication — phase d'accueil

Animé par mistral, cet agent explique les concepts et répond aux questions de clarification de l'étudiant avant le codage. Sa règle cardinale est de ne jamais fournir de code, même à titre d'exemple, et de guider l'étudiant exclusivement par des questions. Un extrait de son prompt système illustre cette contrainte :

```
STRICT RULES:
1. NEVER write, suggest, or complete HTML/CSS/JS code
2. NEVER show code snippets, even as examples
3. Guide them to discover the right tags themselves
4. ALWAYS ask at least one guiding question
5. Keep responses concise (3-6 sentences) and encouraging
```

Outils : *explain_tp_step, get_learning_objectives, structure_clarification, detect_misconception.*

4.3.2. Agent Indices — phase de codage

Animé par deepseek-coder:6.7b (température basse de 0,3 pour la précision technique), cet agent analyse le code HTML en cours d'écriture et délivre des indices progressifs répartis sur quatre niveaux — abstrait, conceptuel, structurel puis spécifique — sans jamais rédiger le code complet. Il valide également les balises HTML requises et conserve l'historique des indices déjà donnés afin de toujours proposer le niveau suivant.

Outils : *validate_html_tags, analyze_html_structure, generate_progressive_hint, assess_hint_level.*

4.3.3. Agent Évaluation — évaluation post-session

Animé par gemma4:31b, cet agent génère de trois à cinq questions à choix multiples ancrées dans le code réellement soumis par l'étudiant, puis évalue les réponses et produit un score accompagné d'un retour personnalisé. Le calcul du score est déterministe (réalisé par un outil dédié), tandis que la synthèse du feedback est confiée au modèle de langage — une répartition qui garantit l'équité de la note tout en conservant la richesse du commentaire.

Outils : *build_quiz_generation_prompt, validate_quiz_structure, calculate_quiz_score, generate_feedback_summary.*

4.3.4. Orchestrateur — routage multi-étapes

Animé par deepseek-v3.2, l'orchestrateur traite les requêtes couvrant plusieurs agents. Ses outils sont des appels HTTP (via la bibliothèque httpx) vers les agents en aval, assortis de délais d'expiration adaptés à chaque tâche — jusqu'à 180 secondes pour la génération de quiz, qui est l'opération la plus coûteuse.

Outils : `call_explanation_agent`, `call_hint_agent`, `call_evaluation_agent_quiz`, `call_evaluation_agent_score`.

4.4. Backend microservices (Spring Cloud)

Le backend transactionnel repose sur cinq services Spring enregistrés auprès d'Eureka, ce qui leur permet de se découvrir mutuellement par leur nom logique plutôt que par adresse codée en dur :

- **Auth Service (8081)** — authentification JWT, hachage BCrypt des mots de passe, CRUD utilisateurs et gestion des rôles TEACHER / STUDENT (classes JwtService, JwtAuthFilter, SecurityConfig).
- **TP Service (8082)** — gestion des TP, des assignations et de la progression, avec conversion JSON assurée par un JsonConverter dédié.
- **API Gateway (8080)** — Spring Cloud Gateway dotée d'un filtre AuthFilter qui valide le JWT et route l'ensemble du trafic `/api/**`.
- **Discovery Server (8761)** — serveur Eureka, registre central de tous les services.

4.5. Agent Gateway (FastAPI)

L'Agent Gateway est le point d'entrée public de la couche IA. Il expose une route dédiée pour chaque interaction et relaie la requête vers le bon agent. Il héberge également la génération de TP assistée par IA et la gestion de la base documentaire de cours (RAG).

Méthode	Route	Rôle
POST	<code>/api/agents/explain</code>	Explication d'une étape.
POST	<code>/api/agents/hint</code>	Indice progressif.
POST	<code>/api/agents/generate-quiz</code>	Génération du quiz.
POST	<code>/api/agents/evaluate</code>	Correction et calcul du score.
POST	<code>/api/agents/orchestrate</code>	Routage multi-agents.
POST	<code>/api/agents/generate-tp</code>	Génération d'un TP complet par l'IA.
GET / POST	<code>/api/agents/courses</code>	Liste / dépôt de documents de cours (RAG).
GET	<code>/agents/health</code>	État de santé de tous les agents.

4.6. Frontend Next.js

L'interface est une application Next.js 14 en TypeScript, stylée avec TailwindCSS selon le thème Catppuccin Mocha. Elle s'organise autour de deux espaces distincts — étudiant et enseignant. Les composants IA (ExplanationChat, HintBox, QuizComponent) consomment l'Agent Gateway via un client typé `agentService.ts`, tandis que `tpService.ts` gère les TP et la progression.

L'espace de travail étudiant repose sur un composant IDELayout combinant un éditeur de code, un aperçu HTML en direct affiché dans une iframe sandboxée (pour isoler le code exécuté), et une boîte à indices. Le patron Interprète, appliqué côté client, valide les balises requises avant toute soumission.

4.7. Mécanismes anti-triche

L'intégrité académique est assurée par plusieurs garde-fous combinés, agissant à la fois au niveau de l'éditeur et au niveau de la session. Aucun mécanisme n'est suffisant seul ; c'est leur combinaison qui rend le contournement coûteux.

Mécanisme	Mise en œuvre
Copier-coller désactivé	onPaste → e.preventDefault() sur l'éditeur (CodeEditor.tsx).
Glisser-déposer désactivé	onDrop → e.preventDefault().
Clic droit désactivé	onContextMenu → e.preventDefault().
Validation par interprète	TagExpression + AndExpression vérifient les balises requises.
Minuteur de session	Égrené par étape, avec sauvegarde automatique toutes les 5 secondes.
Suivi des indices	Le compteur d'indices est persisté dans la progression de l'étudiant.

4.8. Infrastructure et déploiement

L'ensemble des douze services est décrit dans un fichier docker-compose.yml (Compose v2) partageant un réseau commun et doté de contrôles de santé. Un script start.sh permet en outre de lancer toute la pile en une seule commande, sans Docker, dans un environnement de développement : il vérifie les prérequis, contrôle la présence des modèles Ollama, crée un environnement Python partagé, libère les ports nécessaires, démarre les services puis interroge leurs points /health jusqu'à ce qu'ils soient disponibles.

Six topics Kafka sont définis pour un pipeline asynchrone optionnel : agent.explain.requests, agent.hint.requests, agent.evaluation.requests, agent.responses, progress.updates et quiz.completed. Ils préparent une montée en charge future où les agents communiqueraient par messages plutôt que par appels HTTP synchrones.

```

Spring Boot Microservices:
● discovery-server → http://localhost:8761
● auth-service → http://localhost:8081
● tp-service → http://localhost:8082
● api-gateway → http://localhost:8080

Frontend:
● frontend → http://localhost:3000

Open your browser: http://localhost:3000
Agent Gateway API: http://localhost:8000
Agents health: http://localhost:8000/agents/health
Eureka dashboard: http://localhost:8761
API Gateway: http://localhost:8080

Logs: /Users/badrmac/work/mywork/enset-challenge-submission-embedding3x/logs/
To tail a log: tail -f logs/explanation-agent.log

Press Ctrl+C to stop all services

```

Figure 8 — Tableau de bord de lancement (start.sh) : services et ports

5. Réalisation

Cette section présente les fonctionnalités effectivement développées, illustrées par des captures d'écran issues de la démonstration en ligne. Elle suit l'ordre naturel d'utilisation : l'accueil et l'authentification, puis l'espace enseignant et enfin l'espace étudiant. Elle se conclut par les scénarios d'usage et un état d'avancement détaillé.

5.1. Page d'accueil et authentification

La page d'accueil publique présente la plateforme et son positionnement. L'écran de connexion propose des comptes de démonstration distincts pour l'enseignant et l'étudiant, ce qui facilite la prise en main lors de la présentation.

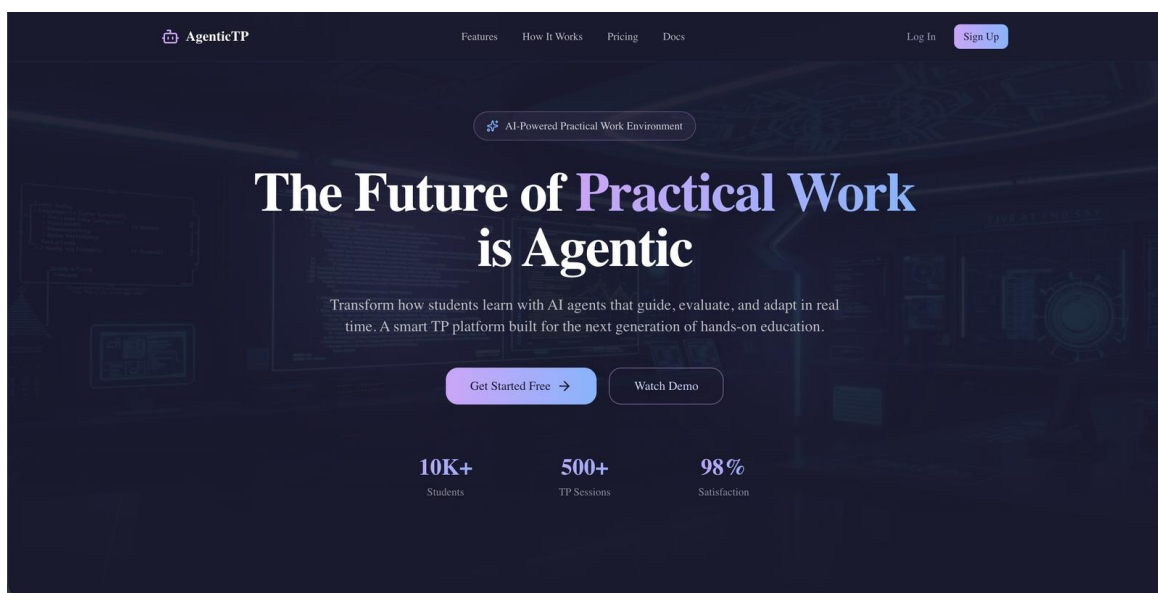


Figure 9 — Page d'accueil publique

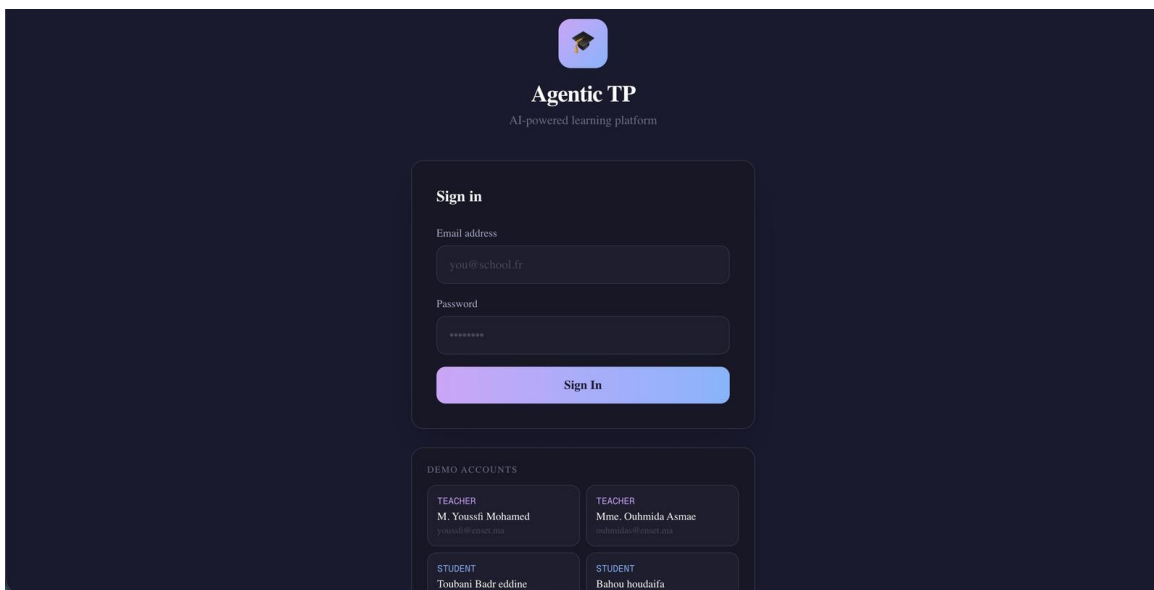


Figure 10 — Écran de connexion avec comptes de démonstration (enseignant / étudiant)

5.2. Espace enseignant

Depuis son tableau de bord, l'enseignant visualise les TP qu'il a créés ainsi que la progression de ses étudiants, et accède aux actions de création et d'assignation. C'est le centre de pilotage pédagogique de la plateforme.

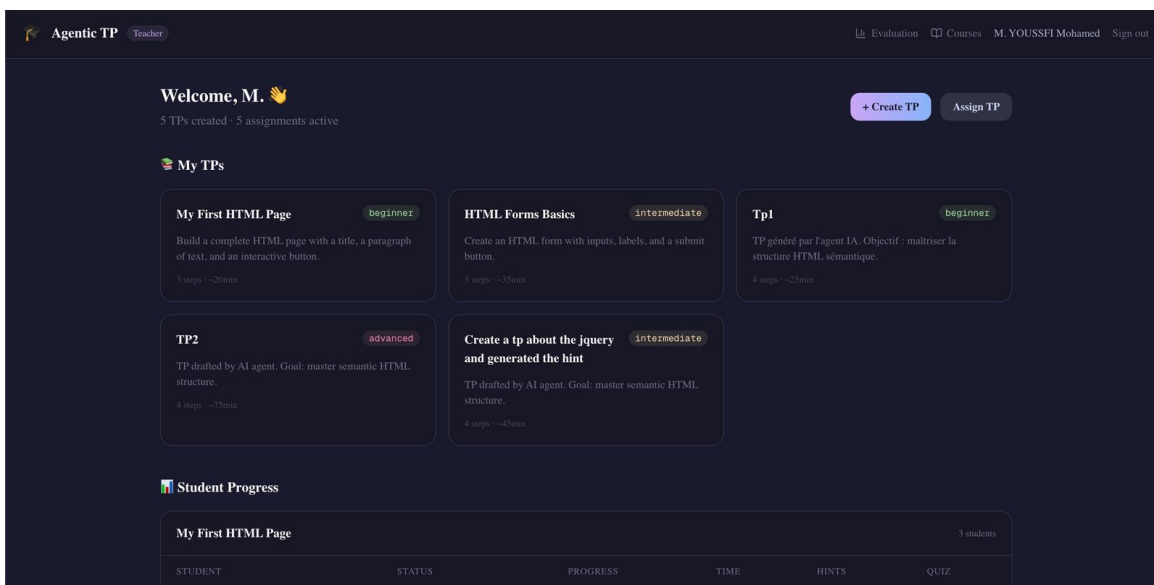


Figure 11 — Tableau de bord enseignant : TP créés et progression des étudiants

La création d'un TP propose deux modes complémentaires : la génération assistée par l'agent IA, qui produit un TP complet à partir d'un énoncé déposé, ou la saisie manuelle, qui laisse à l'enseignant le contrôle total du contenu.

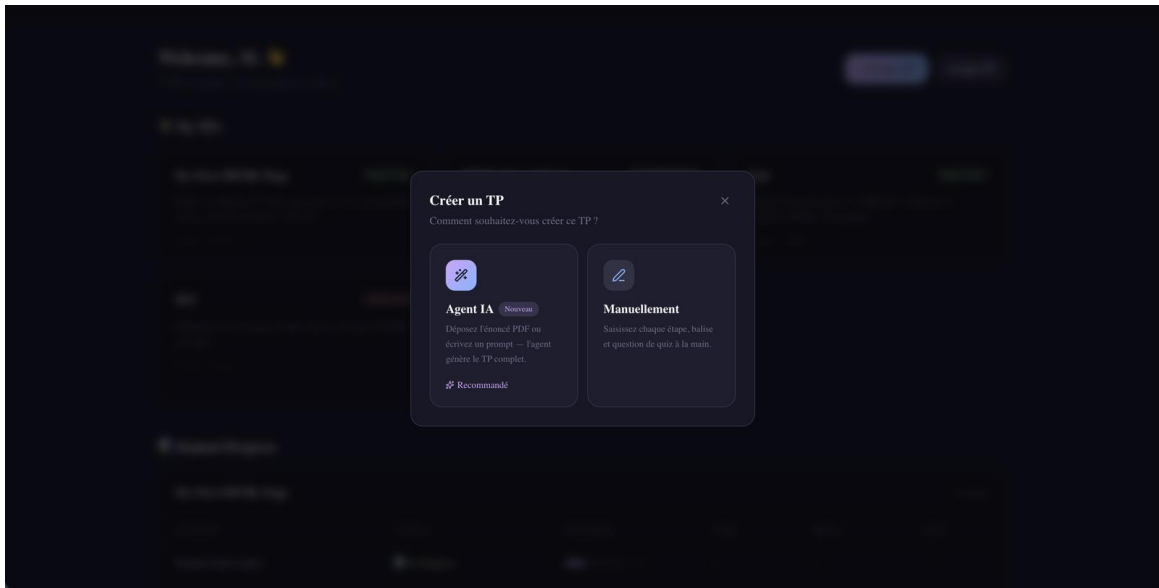


Figure 12 — Choix du mode de création d'un TP

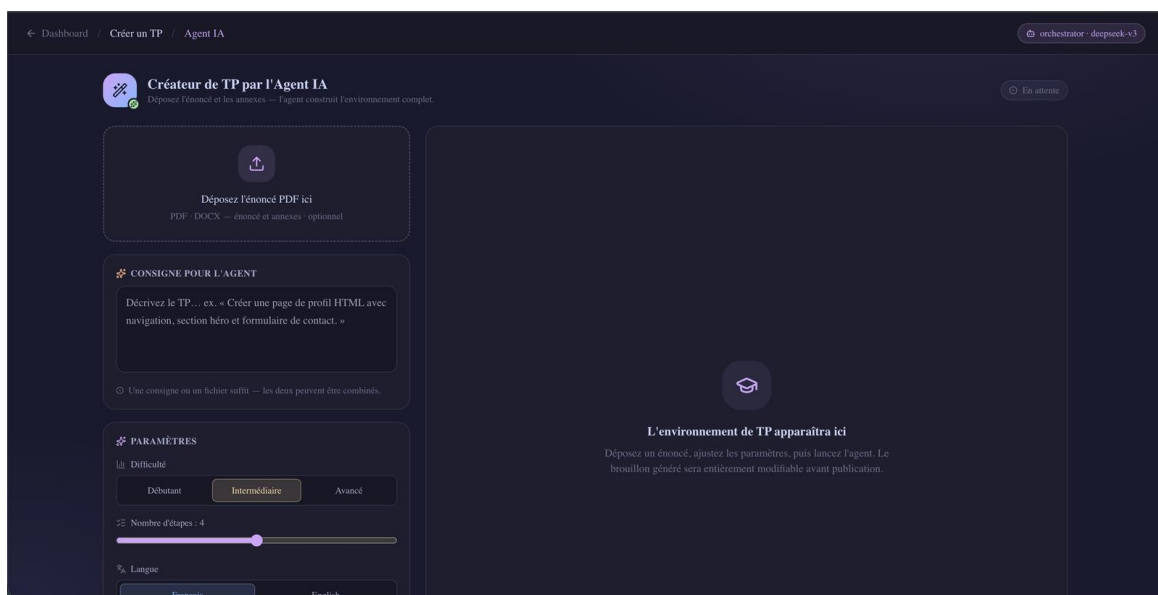


Figure 13 — Créateur de TP par l'agent IA : dépôt d'énoncé, consigne et paramètres

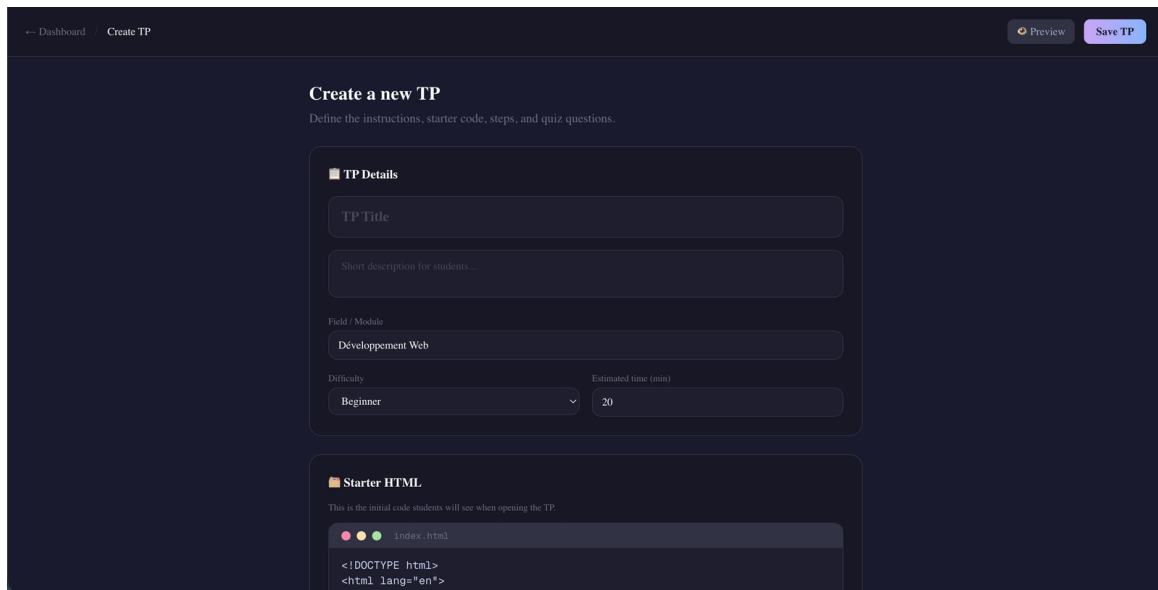


Figure 14 — Création manuelle d'un TP : détails et code de départ

L'enseignant peut alimenter une base documentaire (cours, annexes) indexée, qui sert à contextualiser la génération de contenu par les agents (RAG, génération augmentée par la recherche).

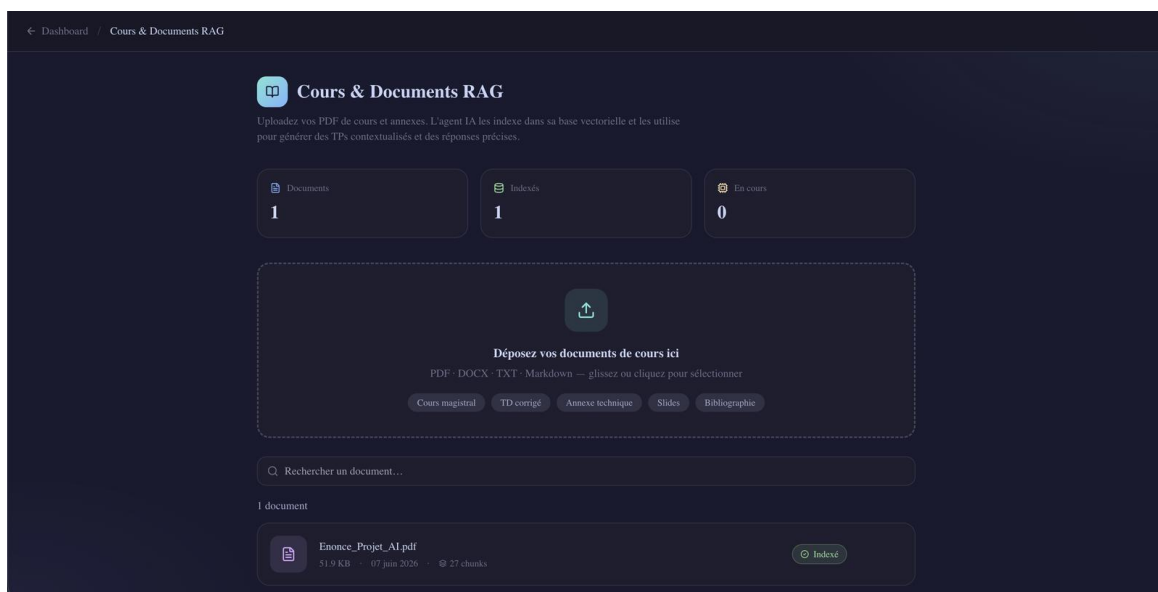


Figure 15 — Cours & Documents RAG : indexation des supports pédagogiques

Enfin, l'espace d'évaluation présente, pour chaque TP et chaque étudiant, le statut d'avancement, la progression détaillée, le temps passé, le nombre d'indices utilisés et le score obtenu — autant d'indicateurs qui permettent à l'enseignant de repérer rapidement les étudiants en difficulté.

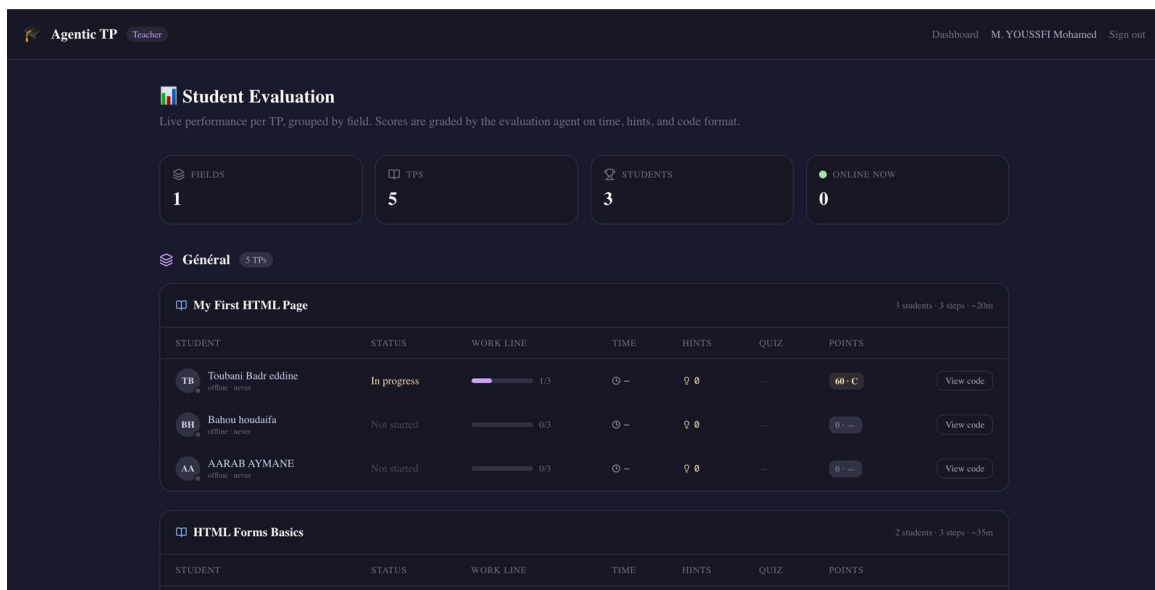


Figure 16 — Évaluation des étudiants : performance en direct par TP

5.3. Espace étudiant

L'étudiant accède d'abord à son tableau de bord, qui liste ses TP assignés et son avancement sur chacun.

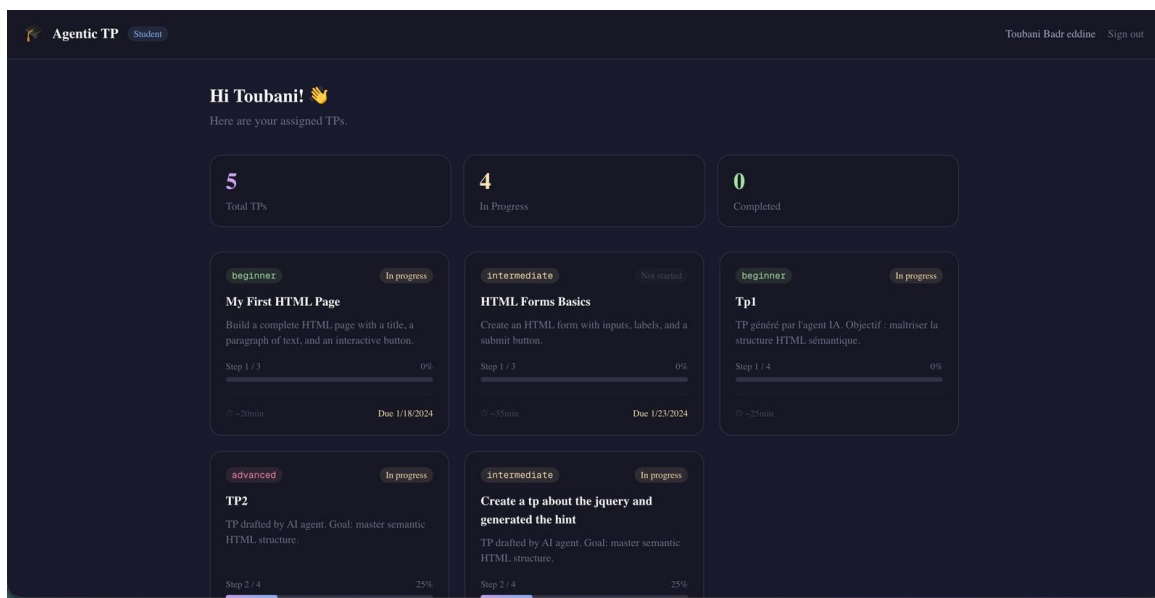


Figure 17 — Tableau de bord étudiant : TP assignés et avancement

Avant de coder, l'étudiant reçoit l'explication de l'étape par l'agent Explication (Mistral) et peut lui poser des questions de clarification — sans jamais obtenir de code en retour.

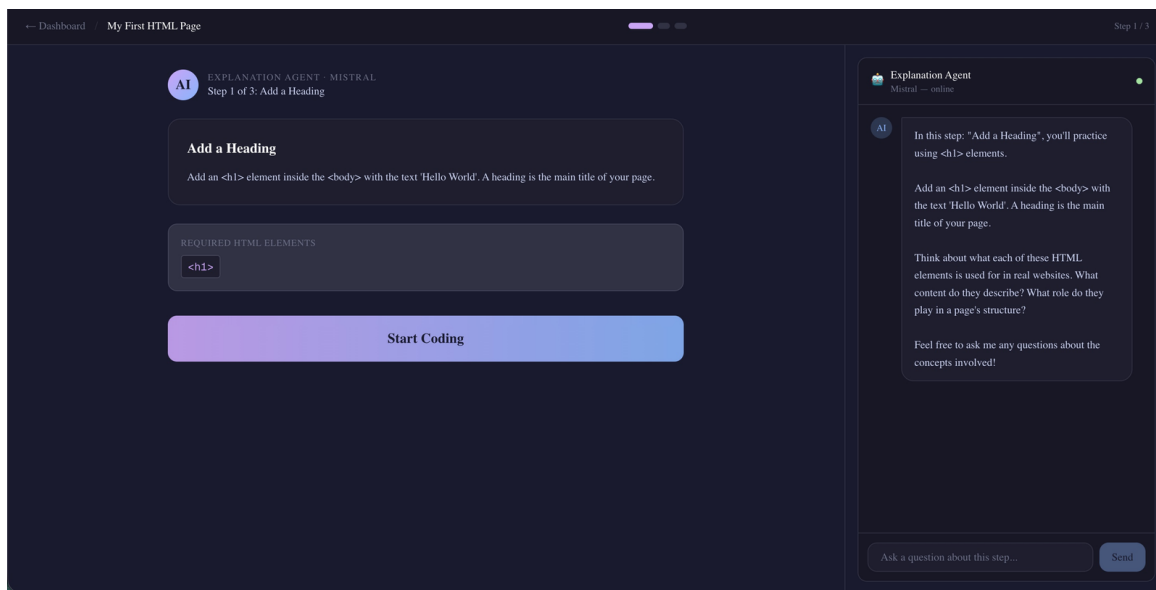


Figure 18 — Phase d'explication : dialogue avec l'agent Explication (Mistral)

L'espace de codage réunit l'éditeur, l'aperçu en direct du rendu, le minuteur de session, le système d'indices et la mention rappelant que le collage est désactivé. C'est ici que se déroule la majeure partie de la session étudiante.

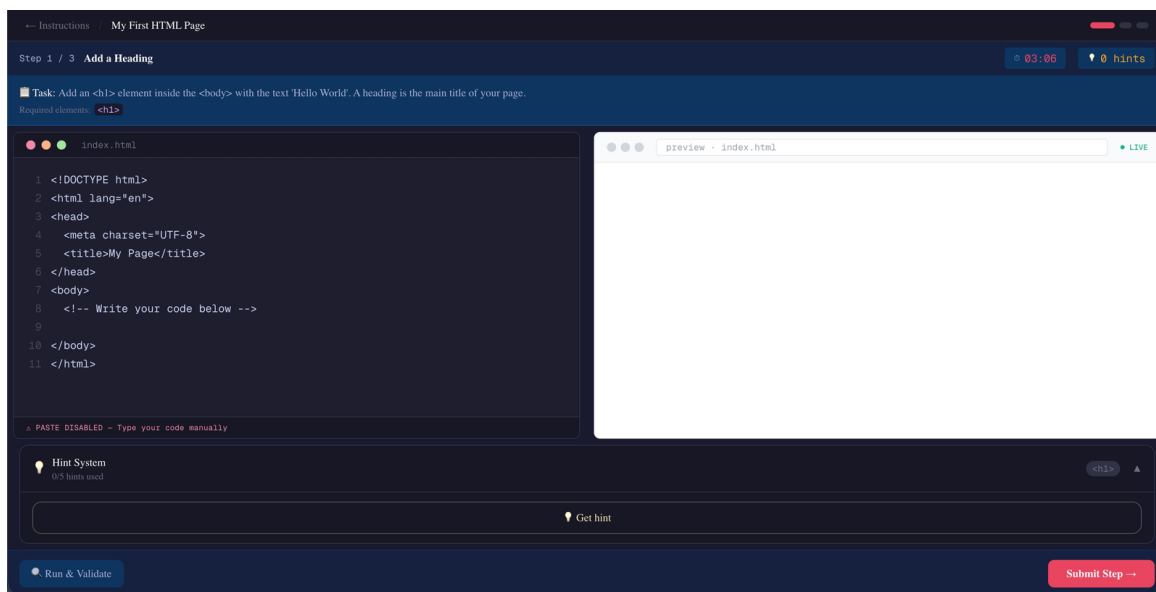


Figure 19 — Espace de codage : éditeur, aperçu live, minuteur et système d'indices

5.4. Scénarios d'utilisation

5.4.1. Parcours étudiant en trois phases

- **Phase 1 — Explication.** L'agent Explication (Mistral) présente l'étape et répond aux questions de l'étudiant. Le collage est désactivé et le minuteur démarre.

- **Phase 2 — Codage.** Dans l'IDE à aperçu live, l'étudiant écrit son code. L'agent Indices (deepseek-coder) fournit des indices progressifs en cas de blocage, et les balises requises sont validées en temps réel.
- **Phase 3 — Évaluation.** L'agent Évaluation (gemma4) génère 3 à 5 QCM à partir du code rendu, corrige les réponses, attribue un score et rédige un retour personnalisé transmis à l'enseignant.

5.4.2. Parcours enseignant

- **Créer un TP** — définir titre, description, difficulté, durée estimée, code de départ, instructions et balises requises par étape, ainsi que les questions de quiz.
- **Assigner** — sélectionner les étudiants concernés et fixer une échéance.
- **Suivre** — consulter le tableau de progression (étapes franchies, temps passé, indices utilisés).
- **Évaluer** — examiner le score au quiz et le feedback généré par l'IA, étudiant par étudiant.

5.5. État d'avancement

La quasi-totalité des fonctionnalités clés est opérationnelle. Le tableau suivant fait le point composant par composant.

Fonctionnalité	État
Frontend — espace étudiant (IDE, aperçu, minuteur)	Terminé
Frontend — tableau de bord enseignant (créer, assigner, suivre)	Terminé
Frontend — composants IA (ExplanationChat, HintBox, QuizComponent)	Terminé
Agent Explication (Mistral + LangGraph)	Terminé
Agent Indices (deepseek-coder:6.7b + LangGraph)	Terminé
Agent Évaluation (gemma4:31b + LangGraph)	Terminé
Orchestrateur (deepseek-v3.2 + LangGraph)	Terminé
Agent Gateway (routeur FastAPI)	Terminé
Auth Service (JWT, BCrypt, Spring Security)	Terminé
TP Service (CRUD, stockage JSON)	Terminé
API Gateway (filtre JWT, routage) + Eureka	Terminé
Docker Compose (12 services) + start.sh + test.sh	Terminé
Mécanismes anti-triche	Terminé
Configuration des topics Kafka	Terminé
Service RAG / pgvector (ingestion des documents)	Partiel

6. Conclusion

6.1. Résumé du projet

La plateforme Agentic TP démontre qu'il est possible de réconcilier accompagnement personnalisé et intégrité académique, deux exigences habituellement opposées. En orchestrant plusieurs agents IA spécialisés — chacun piloté par un modèle de langage local adapté à sa tâche — et en les intégrant à une architecture microservices éprouvée, le projet livre un environnement de TP guidé de bout en bout : explication socratique avant le codage, indices progressifs pendant le codage, et évaluation ancrée dans le code réellement produit par l'étudiant.

L'ensemble est entièrement conteneurisé et lançable d'une seule commande, ce qui en fait une base technique solide et reproductible. Les diagrammes de conception, le découpage en microservices et le patron commun à tous les agents garantissent par ailleurs que la solution reste maintenable et extensible.

6.2. Défis rencontrés et limites actuelles

Le développement a mis en évidence plusieurs défis, dont certains se traduisent par des limites encore présentes dans la version actuelle :

- **Support limité au HTML** — la prise en charge d'autres langages (Python, Java) nécessiterait un exécuteur de code sandboxé, qui n'est pas encore intégré.
- **Suivi enseignant non temps-réel** — la progression est lue au chargement de la page plutôt que poussée par WebSocket, ce qui impose un rafraîchissement manuel.
- **Pipeline RAG incomplet** — la structure du magasin vectoriel existe, mais l'ingestion automatique des documents n'est pas encore branchée.
- **Dépendance aux modèles cloud** — gemma4:31b dans sa variante cloud requiert un compte Ollama cloud ; à défaut, la génération de quiz bascule sur des questions statiques.
- **Authentification frontend** — la page de connexion utilise encore une simulation locale (localStorage) ; le câblage au véritable auth-service reste à finaliser.

6.3. Recommandations et perspectives

Pour faire évoluer et maintenir la solution, les recommandations suivantes sont proposées, classées par horizon temporel.

6.3.1. Court terme

- Brancher le pipeline d'ingestion RAG (Spring AI + pgvector) pour rendre la génération réellement contextualisée.
- Connecter l'authentification frontend au flux JWT réel afin de supprimer la simulation locale.
- Mettre en place un suivi enseignant en temps réel via WebSocket.
- Ajouter une limitation de débit et une mise en cache des appels aux modèles de langage pour maîtriser la charge.

6.3.2. Moyen terme

- Étendre le support à plusieurs langages via un sandbox d'exécution (Judge0 ou Piston).
- Faire communiquer les agents de façon asynchrone par Kafka, en exploitant les topics déjà définis.
- Introduire une détection de similarité de code (anti-plagiat) entre promotions.
- Préparer des manifestes Kubernetes pour un déploiement en production à grande échelle.

6.4. Innovations clés

Le projet se distingue par plusieurs partis pris d'architecture qui constituent ses apports les plus originaux :

- **Un modèle par agent** — chaque agent exécute le LLM optimisé pour sa tâche (modèle de code pour les indices, modèle d'instruction pour les explications, modèle de raisonnement pour l'évaluation et le routage).
- **Appel d'outils LangGraph** — les agents utilisent des outils structurés qui contraignent et fiabilisent leurs sorties, bien au-delà du simple prompting.
- **Repli hors-ligne gracieux** — les composants IA basculent sur une logique déterministe locale dès qu'un agent devient indisponible.
- **Validation par patron Interprète** — un arbre d'expressions composable valide formellement les balises HTML requises.
- **Évaluation ancrée dans le code** — les questions de quiz sont générées à partir du code réellement soumis par l'étudiant, et non d'un contenu générique.

Les perspectives tracées —support multi-langages et déploiement Kubernetes — dessinent un chemin clair vers une solution de production à grande échelle.